

Lab 2

Programs for summation of series $1+X+X^2+X^3+\dots$ with different time complexities.

Approach 1:

```
#include <stdio.h>

int main() {
    double x;
    int n;

    printf("Enter the value of X: ");
    scanf("%lf", &x);
    printf("Enter the number of terms: ");
    scanf("%d", &n);

    double sum = 0;
    double first_term = 1;

    for (int i = 0; i <= n; i++) {
        sum += first_term;
        first_term *= x;
    }

    printf("The summation of the series is: %lf\n", sum);

    return 0;
}
```

output

```
Enter the value of X: 10
Enter the number of terms: 3
The summation of the series is: 1111.000000
```

Approach 2:

```
#include <stdio.h>
#include <math.h>

int main() {
    double x;
    int n;

    printf("Enter the value of X: ");
    scanf("%lf", &x);
    printf("Enter the highest power: ");
    scanf("%d", &n);

    double sum;

    if (x == 1.0) {
        sum = n + 1;
    } else {
        sum = (pow(x, n + 1) - 1.0) / (x - 1.0);
    }

    printf("The summation of the series is: %.2lf\n", sum);

    return 0;
}
```

Output:

```
Enter the value of X: 10
Enter the highest power: 3
The summation of the series is: 1111.00
```

Write a program to find the Kth smallest element in the array.

```

#include <iostream>
#include <vector>
#include <queue>

int findKthSmallest(const std::vector<int>& arr, int k) {
    std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
    for (int num : arr) {
        minHeap.push(num);
    }
    for (int i = 0; i < k - 1; ++i) {
        minHeap.pop();
    }
    return minHeap.top();
}

int main() {
    int n, k;
    std::cout << "Enter the number of elements in the array: ";
    std::cin >> n;

    if (n <= 0) {
        std::cout << "Array size must be greater than 0." << std::endl;
        return 1;
    }
    std::vector<int> arr(n);
    std::cout << "Enter " << n << " integers separated by spaces or newlines:\n";
    for (int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }
    std::cout << "Enter the value of K: ";
    std::cin >> k;

    if (k > 0 && k <= arr.size()) {
        int result = findKthSmallest(arr, k);
        std::cout << "The " << k << "-th smallest element is: " << result << std::endl;
    } else {
        std::cout << "Invalid value of K! K must be between 1 and " << arr.size() << "." << std::endl;
    }

    return 0;
}

```

Output:

```

Enter the number of elements in the array: 5
Enter 1
6
5
28
3
Enter the value of K: 3
The 3-th smallest element is: 5

```

Approach 2:

```

#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int findKthSmallest(int arr[], int n, int k) {
    if (k > 0 && k <= n) {
        bubbleSort(arr, n);
        return arr[k - 1];
    }
    return -1;
}

int main() {
    int n, k, i;

    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 1;
    }

    int arr[n];
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) {
            return 1;
        }
    }

    printf("Enter the value of K: ");
    if (scanf("%d", &k) != 1) {
        return 1;
    }

    int result = findKthSmallest(arr, n, k);

    if (result != -1) {
        printf("%d\n", result);
    } else {
        printf("Invalid K value\n");
    }

    return 0;
}

```

Output:

```
Enter number of elements: 5
Enter 5 elements: 1
6
8
6
4
Enter the value of K: 2
4
```

Create a Binary Search Tree and perform the insertion, deletion operations.

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    }
    return root;
}

struct Node* findMin(struct Node* root) {
    while (root && root->left != NULL) {
        root = root->left;
    }
    return root;
}

struct Node* deleteNode(struct Node* root, int value) {
    if (root == NULL) {
        return root;
    }
    if (value < root->data) {
        root->left = deleteNode(root->left, value);
    } else if (value > root->data) {
        root->right = deleteNode(root->right, value);
    } else {
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
    }
}

```

```

        struct Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct Node* root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 70);
    root = insert(root, 60);
    root = insert(root, 80);

    inorder(root);
    printf("\n");

    root = deleteNode(root, 20);
    inorder(root);
    printf("\n");

    root = deleteNode(root, 30);
    inorder(root);
    printf("\n");

    root = deleteNode(root, 50);
    inorder(root);
    printf("\n");

    return 0;
}

```

Output:

```

20 30 40 50 60 70 80
30 40 50 60 70 80
40 50 60 70 80
40 60 70 80

```